

Online Reinforcement Learning: Settings and Performance Measures

Mohammad Sadegh Talebi

m.shahi@di.ku.dk

Department of Computer Science

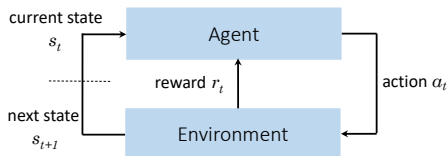


Recap

An **agent** interacts with the MDP for N rounds.

At each time step t :

- The agent observes the current state s_t and takes an action $a_t \in \mathcal{A}$
- MDP generates a reward $r_t \sim R(s_t, a_t)$ and a next state $s_{t+1} \sim P(\cdot | s_t, a_t)$



This interaction produces $h_t = (s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_{t-1}, a_{t-1}, r_{t-1}, s_t)$.

In RL, MDP parameters (P and R) are **unknown** ... yet we wish to learn an optimal policy using **collected experience (data)**.



RL Classification



RL Settings

RL problems (and methods) can be categorized according to:

- problem horizon / objective
- data collection model
- design approach



Classification Based on Horizon/Objective

A classification of RL problems based on horizon (and objective):

- Discounted MDPs $\implies$ Discounted RL problems
- Finite-horizon MDPs (with fixed horizon) $\implies$ Episodic RL problems
- Average-reward MDPs $\implies$ Average-reward RL problems



Discounted RL

The agent interacts with a discounted MDP

$$M = (\mathcal{S}, \mathcal{A}, \underbrace{P, R}_{\text{unknown}}, \gamma)$$

- The interaction proceeds for an arbitrary number of time steps **without any reset**.
- The initial state is chosen by Nature.
- Objective: to learn a policy solving

$$\max_{\pi \in \Pi^{\text{SD}}} V^{\pi}(s) = \mathbb{E}^{\pi} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r_t \mid s_1 = s \right]$$

for any initial state $s \in \mathcal{S}$.

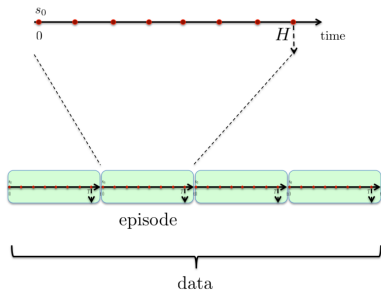


Episodic RL

The agent interacts with a finite-horizon MDP with a fixed horizon H

$$M = (\mathcal{S}, \mathcal{A}, H, \underbrace{\{P_h\}_{h \in [H]}, \{R_h\}_{h \in [H]}}_{\text{unknown}})$$

- The interaction proceeds for an arbitrary number of episodes.
- The length of each episode is H (fixed, known).
- The state is reset at the beginning of each episode.



Episodic RL

The agent interacts with a finite-horizon MDP

$$M = (\mathcal{S}, \mathcal{A}, H, \underbrace{\{P_h\}_{h \in [H]}, \{R_h\}_{h \in [H]}}_{\text{unknown}})$$

- The interaction proceeds for an arbitrary number of episodes.
- The length of each episode is H (fixed, known).
- The state is reset at the beginning of each episode.
- Objective: to learn a policy solving

$$\max_{\pi \in \Pi^{\text{MD}}} V^{\pi}(s, 1)$$

for any initial state $s \in \mathcal{S}$.



Average-Reward RL

The agent interacts with an average-reward MDP

$$M = (\mathcal{S}, \mathcal{A}, \underbrace{P, R}_{\text{unknown}})$$

- The interaction proceeds for an arbitrary number of time steps **without any reset**.
- The initial state of each episode is chosen by Nature.
- Objective: to learn a policy solving

$$\max_{\pi \in \Pi^{\text{SD}}} g^{\pi}(s) = \lim_{N \rightarrow \infty} \frac{1}{N} \mathbb{E}^{\pi} \left[\sum_{t=1}^N r_t \mid s_1 = s \right]$$

for any initial state $s \in \mathcal{S}$.

- The largest of class of meaningfully learnable MDPs is that of **weakly communicating** MDPs, for which $g^*(s) = g^*$.



Classification Based on Data Collection

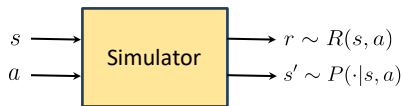
A classification of RL problems based on data collection model:

- **Simulator model:** A (black-box) simulator of the MDP is available.
- **Offline (Batch):** works with pre-collected data using some behavior policy — closely resembles supervised ML.
- **Online RL:** learning from scratch with no training data, and data is collected along the way.



Simulator Model

A **simulator** (or **generative model**) of an MDP M is a device that generates, for any queried input (s, a) , $r \sim R(s, a)$ and $s' \sim P(\cdot|s, a)$.

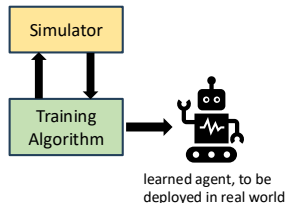


- Input state-action (s, a) can be chosen *arbitrarily*.
- Making N calls to the simulator with input (s, a) gives

$$\{(r_i, s'_i)\}_{i=1}^N \quad \text{with} \quad r_i \sim_{\text{i.i.d.}} R(s, a) \quad s'_i \sim_{\text{i.i.d.}} P(\cdot|s, a)$$

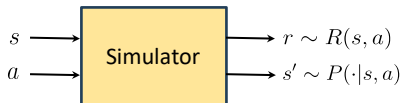
RL under simulator access.

- The Algorithm collects data from the simulator (**uniform** vs **adaptive sampling**)
- Then it computes a near-optimal policy



Simulator Model

A **simulator** (or **generative model**) of an MDP M is a device that generates, for any queried input (s, a) , $r \sim R(s, a)$ and $s' \sim P(\cdot|s, a)$.



- Input state-action (s, a) can be chosen *arbitrarily*.
- Making N calls to the simulator with input (s, a) gives

$$\{(r_i, s'_i)\}_{i=1}^N \quad \text{with} \quad r_i \sim_{\text{i.i.d.}} R(s, a) \quad s'_i \sim_{\text{i.i.d.}} P(\cdot|s, a)$$

How is it built?

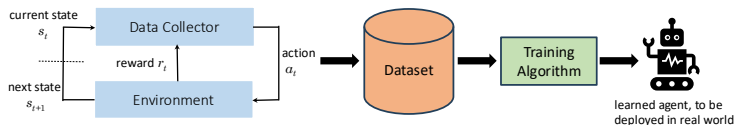
- **rule-based simulators:** domain knowledge, laws from physics chemistry, finance ...
- **learned simulators:** models built using rich datasets



Offline vs. Online

Offline RL. Given a pre-collected dataset from MDP, find a near-optimal policy.

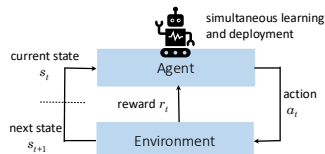
- Benefits from training data, but challenging due to **lack of exploration**.



Online RL. Find a near-optimal policy only using the ever-growing experience:

$$h_t = (s_1, a_1, r_1, \dots, s_{t-1}, a_{t-1}, r_{t-1}, s_t)$$

- Learning from scratch (no training data)
- Challenge lies in balancing **exploration** and **exploitation**.



Classification Based on Design Approaches

Three main approaches to algorithm design in RL:

- **Model-Based:** Consists in maintaining an approximate MDP model through estimating R and P , and deriving a Q-value function from the approximate MDP.
 - Examples: CE-based methods (CE-OPO, CE-OPE), UCRL2 (next lecture).
- **Model-Free:** Directly learns a value function (without estimating R and P), and derives a policy from it.
 - Examples: TD, variants of Q-Learning, DQN.
- **Policy-Based:** Directly searches in the space of policies.
 - Examples: Policy Gradient, PPO.

More recent terminology: **model-based** vs. **valued-based** vs. **policy-based**



Performance Criteria in Online RL



Online RL: Performance Measures

Many offline algorithms can be made online with some tricks (e.g., QL) ... but will they explore well?

For online RL, we need ways to assess the quality of **exploration-exploitation** tradeoff.

The performance of a learning algorithm $\mathbb{A}$ can be measured through:

- **Convergence:** Whether $\mathbb{A}$ converges to an optimal (or near-optimal) policy.
- **Sample Complexity:** The number of steps where the value of the current policy output by $\mathbb{A}$ is not near-optimal.
- **Regret:** The amount of reward lost due to choosing sub-optimal actions by $\mathbb{A}$.



Sample Complexity of Exploration

Consider an RL algorithm $\mathbb{A}$, and $h = (s_1, a_1, r_1, \dots, s_t, a_t, r_t, \dots)$ a trajectory of $\mathbb{A}$, with $a_t \sim \pi_t(\cdot|s_t)$. I.e., π_t is the learned policy at time t .

A notion of sample complexity introduced by (Kakade, 2003):

Sample Complexity of Exploration

For input $\varepsilon > 0$, time step t in h is **bad** if π_t is not ε -optimal for the current state s_t :

$$V^{\pi_t}(s_t) < V^*(s_t) - \varepsilon \implies t \text{ is } \varepsilon\text{-bad}$$

The **sample complexity of exploration** of $\mathbb{A}$ w.r.t. h is the total number of **ε -bad time steps** over the entire trajectory:

$$\sum_{t=1}^{\infty} \mathbb{I}\{V^{\pi_t}(s_t) < V^*(s_t) - \varepsilon\}$$

- It measures the number of mistakes along the *whole trajectory*.
- Sample complexity can be used as a relevant performance measure in discounted RL problems.



PAC-MDP Algorithms

We are interested in RL algorithms, whose sample complexity of exploration are controlled by some functions that are not too large as a function of relevant parameters $S, A, \varepsilon, \delta$ and γ .

A notion of PAC-MDP Algorithm was developed in (Strehl et al., 2005):

PAC-MDP Algorithm

An algorithm $\mathbb{A}$ is called **(ε, δ) -PAC-MDP** if for any ε and δ , the sample complexity of exploration under $\mathbb{A}$ is upper bounded w.p. $\geq 1 - \delta$ by some polynomial in

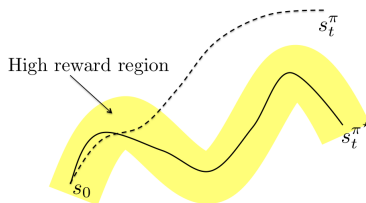
$$S, A, \frac{1}{\varepsilon}, \frac{1}{\delta}, \text{ and } \frac{1}{1-\gamma}.$$

PAC-MDP $\equiv$ Probably Approximately Correct in MDPs



An Important Remark

Defining Sample Complexity *w.r.t. the trajectory of the algorithm* is not always meaningful:



$$\mathbb{P}\left(V^{\pi_t}(\mathbf{s}_t) \geq V^*(\mathbf{s}_t) - \varepsilon\right) \geq 1 - \delta$$

Due to exploration, we can end up in states with very low rewards, and **being optimal from there may not mean much**. A more meaningful criterion would be

$$\mathbb{P}\left(V^{\pi_t}(\mathbf{s}_t) \geq V^*(\mathbf{s}_t^*) - \varepsilon\right) \geq 1 - \delta$$



where $\mathbf{s}_t^*$ is the state on the trajectory of an optimal agent.

Regret: Average-Reward RL

In online average-reward RL, the **Regret** of an algorithm $\mathbb{A}$ is the difference between cumulative reward of the optimal policy π^* (oracle) and that gathered by $\mathbb{A}$:

$$\mathfrak{R}(\mathbb{A}, T) := \sum_{t=1}^T \mathbf{r}_t^* - \sum_{t=1}^T \mathbf{r}_t$$

- Agent $\mathbb{A}$'s trajectory:

$$\forall t: \quad a_t = \mathbb{A}(h_t), \quad r_t \sim R(\mathbf{s}_t, a_t), \quad \mathbf{s}_{t+1} \sim P(\cdot | \mathbf{s}_t, a_t)$$

- Oracle's trajectory:

$$\forall t: \quad r_t^* \sim R(\mathbf{s}_t^*, \pi^*(\mathbf{s}_t^*)), \quad \mathbf{s}_{t+1}^* \sim P(\cdot | \mathbf{s}_t^*, \pi^*(\mathbf{s}_t^*))$$

Regret is directly connected to the agent's goal (maximizing $\sum_{t=1}^T r_t$).
Alternatively, the objective of the agent is to **minimize the regret**.

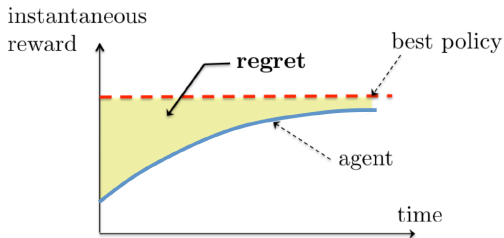


Regret: Average-Reward RL

In online average-reward RL, the **Regret** of an algorithm $\mathbb{A}$ is the difference between cumulative reward of the optimal policy π^* (oracle) and that gathered by $\mathbb{A}$:

$$\mathfrak{R}(\mathbb{A}, T) := \sum_{t=1}^T r_t^* - \sum_{t=1}^T r_t$$

Regret is directly connected to the agent's goal (maximizing $\sum_{t=1}^T r_t$).
Alternatively, the objective of the agent is to **minimize the regret**.



No-Regret Algorithm

- $\mathfrak{R}(\mathbb{A}, T)$ is a r.v., and we wish to control it **in expectation** or **its deviation with high probability**.
- An algorithm $\mathbb{A}$ is a **no-regret learning algorithm** if either

$$\mathbb{E}[\mathfrak{R}(\mathbb{A}, T)] = o(T) \quad \text{or} \quad \mathfrak{R}(\mathbb{A}, T) = o(T) \quad \text{w.h.p.}$$

No-Regret Algorithm

An algorithm $\mathbb{A}$ is called **no-regret** if there exists a deterministic function f with $\frac{f(T)}{T} \rightarrow_{T \rightarrow \infty} 0$, such that one of the following holds:

$$\mathbb{E}[\mathfrak{R}(\mathbb{A}, T)] \leq f(T)$$

$$\mathfrak{R}(\mathbb{A}, T) \leq f(T) \quad \text{with high probability.}$$

- f could be MDP-dependent but must be deterministic.
- Note that a high-probability bound on $\mathfrak{R}(\mathbb{A}, T)$ implies a bound on $\mathbb{E}[\mathfrak{R}(\mathbb{A}, T)]$, but not the other way around.



Regret: Episodic RL

In online episodic RL, the **Regret** of an algorithm $\mathbb{A}$ is the difference between cumulative reward of the optimal policy π^* (oracle) and that gathered by $\mathbb{A}$:

$$\mathfrak{R}(\mathbb{A}, K) := \sum_{k=1}^K V^*(\mathbf{s}_{1,k}, 1) - \sum_{k=1}^K V^{\pi_k}(\mathbf{s}_{1,k}, 1)$$

- π_k is the agent $\mathbb{A}$'s policy executed in episode k :

$$\pi_k = \mathbb{A}(h_k)$$

where h_k denotes the history up to beginning of episode k .

- Both oracle and $\mathbb{A}$ are reset to a fixed initial state at the beginning of each episode, i.e.,

$$s_{1,k} = s_{1,k}^*$$

- When each episode is of fixed length H , we have $T = KH$.

Regret is directly connected to the agent's goal, namely maximizing

$$V^{\pi}(s_1, 1) = \sum_{h=1}^H r(s_h, a_h) + r_H(s_H)$$

Alternatively, the objective of the agent is to **minimize the regret**.



References

- Sham Kakade, "On the sample complexity of reinforcement learning," *PhD Thesis, University of London, University College London*, 2003.
- Ronen Brafman and Moshe Tennenholtz, "R-max -A general polynomial time algorithm for near-optimal reinforcement learning," *Journal of Machine Learning Research*, 2002.
- Alexander Strehl et al., "PAC model-free reinforcement learning" *International Conference on Machine Learning*, 2006.
- Chi Jin, Zeyuan Allen-Zhu, Sebastien Bubeck, and Michael Jordan, "Is Q-learning provably efficient?" *Advances in Neural Information Processing Systems 31*, 2018.
- Kefan Dong, Yuanhao Wang, Xiaoyu Chen, and Liwei Wang, "Q-learning with UCB exploration is sample efficient for infinite-horizon MDP" *International Conference on Learning Representations*, 2020.
- Tor Lattimore and Marcus Hutter, "Near-optimal PAC bounds for discounted MDPs" *Theoretical Computer Science*, 2014.
- Eyal Even-Dar and Yishay Mansour, "Convergence of optimistic and incremental Q-learning," *Advances in Neural Information Processing Systems 14*, 2001.

